Taller: Aplicación de Patrones de Diseño

Un módulo por grupo (8 módulos) – NestJS + React + PostgreSQL

1) Objetivo

Construir una solución académica modular donde **cada equipo entrega un único módulo funcional** (MVP del módulo), aplicando patrones de diseño (creacional, estructural y de comportamiento) y garantizando integración técnica mínima (API + DB + UI simple).

2) Equipos

- **Grupo 1:** 3 estudiantes
- **Grupo 6:** 3 estudiantes
- **Grupos 2,3,4,5,7,8:** 4 estudiantes

3) Exposiciones

Cada estudiante debe exponer un patrón de diseño escogido en su grupo.

Regla de patrones

- **Grupos de 4:** 4 patrones (1 creacional + 1 estructural + 2 de comportamiento **o** 2 estructurales + 1 comportamiento)
- **Grupos de 3 (1 y 6):** 3 patrones (1+1+1)

4) Stack obligatorio

- **TypeScript**
- **Backend:** NestJS (REST)
- **Frontend:** React
- **DB:** PostgreSQL con Prisma o Supabase **solo DB/ORM** (sin Supabase Auth)
- **GitHub:** ramas, PR, issues, README

5) Alcance mínimo por módulo (para todos)

Cada módulo debe incluir:

- **3–5 entidades** (tablas)
- **6 endpoints mínimo**
    - 4 CRUD + 2 consultas (filtros/estado/rangos)
- **1 flujo UI completo** (listar + crear/actualizar + acción clave)
- **Validación + manejo de errores**

6) Módulo asignado por grupo (usando tus ideas)

Importante: **NO es toda la plataforma**, solo **un módulo central** de ese negocio.

Grupo 1 (3 estudiantes)

Módulo: "Préstamos de Dispositivos" (instituciones educativas)

Qué hace: registrar préstamo, devolución, disponibilidad y vencimientos.

Patrones (3 obligatorios):

- **Creacional:** Factory Method (crear préstamo según tipo de dispositivo)
- **Composite:** permite tratar un dispositivo individual y un kit o paquete como si fueran el mismo tipo de elemento prestable.
- **Comportamiento:** State (solicitado → aprobado → entregado → devuelto → vencido)

Grupo 2 (4 estudiantes)

Módulo: "Motor de Recomendaciones por Escaneo" (IA/RA)

Qué hace: registrar un "scan", generar alternativas de uso y guardar historial + feedback.

Patrones (4 obligatorios):

- **Creacional:** Builder (construir RecommendationResult)
- **Flyweight:** reutiliza datos compartidos (ej. categorías, tipos de transformación, etiquetas) para optimizar memoria.

- **Comportamiento:** Strategy (tipos de recomendación: reutilizar/transformar/reconfigurar)
- **Comportamiento:** Observer (notificar cuando hay resultados parciales/finales)

Grupo 3 (4 estudiantes)

Módulo: "Reclamación y Verificación de Objetos" (Lost & Found Uninorte)

Qué hace: flujo para reclamar objeto y validar evidencias hasta aprobación/rechazo.

Patrones (4 obligatorios):

- **Creacional:** Abstract Factory (crear flujo por tipo de objeto/documento)
- **Estructural:** Proxy (proteger acceso a evidencias/datos sensibles)
- **Comportamiento:** Chain of Responsibility (validaciones encadenadas de reclamación)
- **Visitor:** permite aplicar distintas operaciones (auditoría, generación de reporte, cálculo de coincidencia) sobre las evidencias sin modificar sus clases.

Grupo 4 (4 estudiantes)

Módulo: "Publicación Social Musical" (SoundShare)

Qué hace: publicar estado emocional asociado a canción + reacciones + privacidad.

Patrones (4 obligatorios):

- **Creacional:** Prototype (plantillas de publicación por mood)
- **Estructural:** Decorator (agregar reacciones/tags/privacidad sin inflar la clase base)
- **Comportamiento:** Mediator (coordinar post–comentarios–notificaciones)
- **Memento:** guarda y restaura versiones de una publicación (ej. antes de editar) para permitir deshacer cambios

Grupo 5 (4 estudiantes)

Módulo: “Contratación de Servicio” (marketplace de trabajos cortos)

Qué hace: publicar solicitud, aceptar, ejecutar, finalizar, cancelar/reprogramar.

Patrones (4 obligatorios):

- **Creacional:** Factory Method (crear orden por tipo: horas/tarea/paquete)
- **Estructural:** Facade (contratar servicio: validar→cotizar→orden)
- **Comportamiento:** Command (cancelar/reprogramar/confirmar como comandos)
- **Template Method:** define el “paso a paso” fijo del proceso de contratación, permitiendo variaciones por tipo de servicio en pasos específicos.

Grupo 6 (3 estudiantes)

Módulo: “Lecturas y Alertas de Tanques” (ondas sonoras)

Qué hace: registrar lecturas, calibración básica, alertas por umbral.

Patrones (3 obligatorios):

- **Creacional:** Singleton (config/calibración global)
- **Estructural:** Adapter (adaptar sensores/protocolos simulados)
- **Comportamiento:** Observer (alertas por cambios de nivel/umbrales)

Grupo 7 (4 estudiantes)

Módulo: “Evaluación de Riesgo por Ingredientes” (SafeCare)

Qué hace: ingresar lista de ingredientes, calcular puntaje y explicar resultado.

Patrones (4 obligatorios):

- **Creacional:** Builder (RiskReport)
- **Estructural:** Proxy (acceso a detalle científico/explicación avanzada)
- **Comportamiento:** Strategy (cálculo de riesgo por perfil: sensible/normal)
- **Comportamiento:** Chain of Responsibility (reglas por ingrediente)

Grupo 8 (4 estudiantes)

Módulo: “Solicitud y Modo de Entrega” (servicio ejecutado vs mentoría)

Qué hace: crear solicitud y elegir modalidad: ejecutado / mentoría / híbrido.

Patrones (4 obligatorios):

- **Creacional:** Abstract Factory (crear flujo según modalidad)
- **Estructural:** Bridge (tipo de servicio vs canal/entrega)
- **Comportamiento:** State (estado de solicitud)
- **Comportamiento:** Iterator (plan/itinerario de mentoría por pasos)

6) Entregables por grupo

1. Código del módulo (NestJS + DB + UI mínima React)
2. Diapositivas de sus exposiciones
3. Documento (1–2 páginas) explicando: problema → diseño → patrones → evidencias en código
4. Video demo (3–6 min) con flujo y evidencia de patrones

Grupo	Módulo	Creacional	Estructural	Comportamiento	Extra (si aplica)
1	Préstamos de dispositivos	**Factory Method**	**Composite**	**State**	—
2	Motor de recomendaciones (IA/RA)	**Builder**	**Flyweight**	**Strategy**	**Observer**
3	Reclamación de objetos perdidos	**Abstract Factory**	**Proxy**	**Chain of Responsibility**	**Visitor**
4	Publicación social musical	**Prototype**	**Decorator**	**Mediator**	**Memento**
5	Contratación de servicio (Marketplace)	**Factory Method**	**Facade**	**Command**	**Template Method**
6	Lecturas y alertas de tanques	**Singleton**	**Adapter**	**Observer**	—
7	Evaluación de riesgo (SafeCare)	**Builder**	**Proxy**	**Strategy**	**Chain of Responsibility**
8	Solicitud y modalidad de servicio	**Abstract Factory**	**Bridge**	**State**	**Iterator**